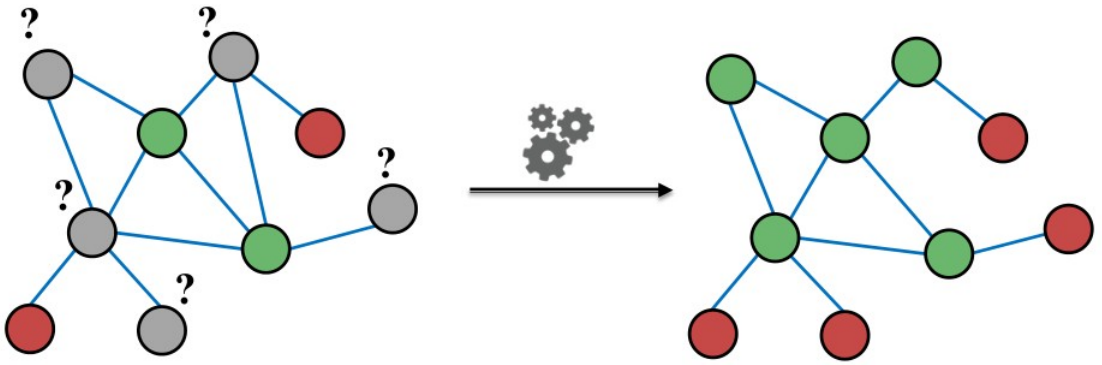
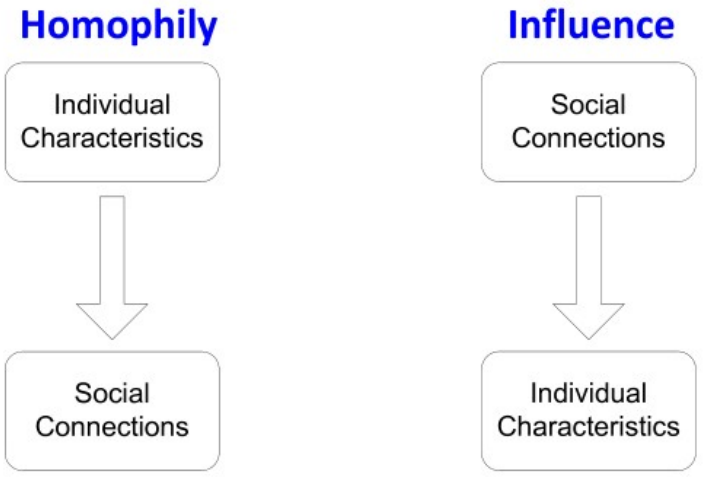
Классификация нод
и
передача сообщений



- Given labels of some nodes
- Let's predict labels of unlabeled nodes
- This is called **semi-supervised node classification**

- **Main question today:** Given a network with labels on some nodes, how do we assign labels to all other nodes in the network?
- **Today we will discuss an alternative framework: Message passing**
- **Intuition: Correlations (dependencies)** exist in networks.
  - In other words: Similar nodes are connected.
  - **Key concept** is **collective classification**: Idea of assigning labels to all nodes in a network together.
- **We will look at three techniques today:**
  - **Relational classification**
  - **Iterative classification**
  - **Correct & Smooth**

Причины корреляции связей нод и свойств нод

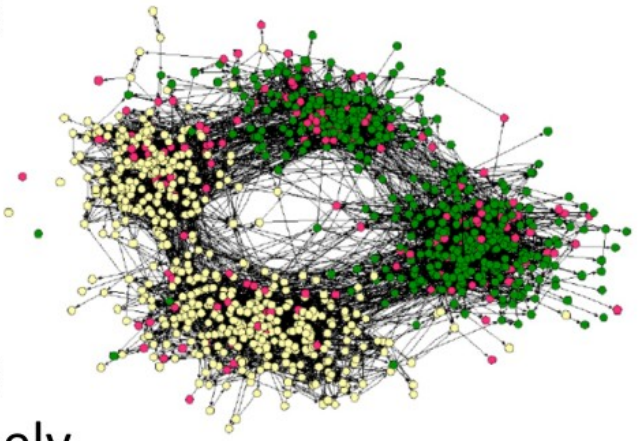


Гомофилия –

тенденция организмов взаимодействовать и группироваться с подобными себе

“Рыбак рыбака видит издалека.”

- Online social network
  - Nodes = people
  - Edges = friendship
  - Node color = interests (sports, arts, etc.)
- People with the same interest are more closely connected due to homophily



(Easley and Kleinberg, 2010)

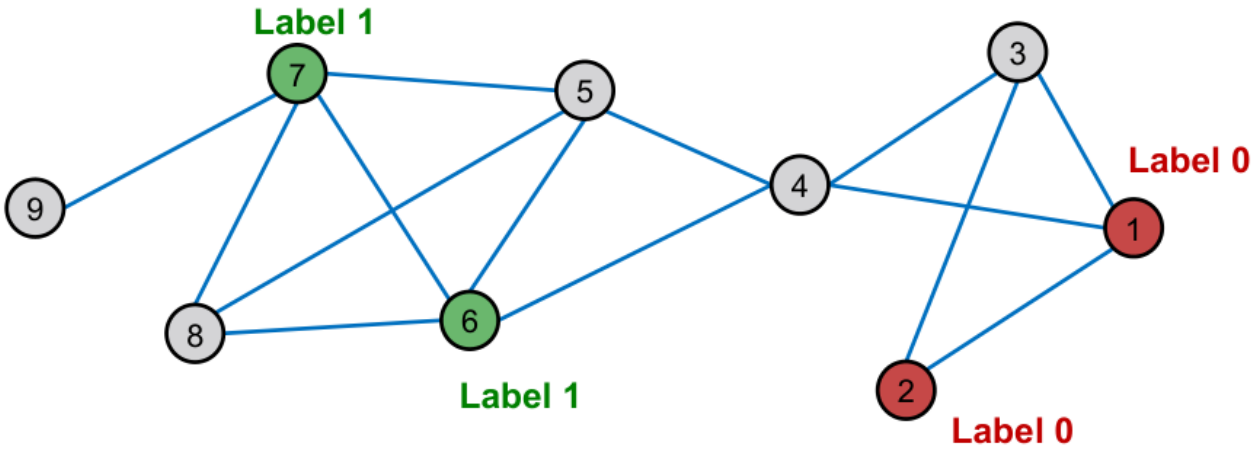
Влияние – тенденция орагнизмов изменяться в зависимости от окружения

“С волками жить – по-волчьи выть”

- **Example**: I recommend my musical preferences to my friends, until one of them grows to like my same favorite genres!

Как воспользоваться корреляцией для прогноза меток нода?

- **How do we leverage this correlation observed in networks to help predict node labels?**



How do we predict the labels for the nodes in grey?

Мотивация

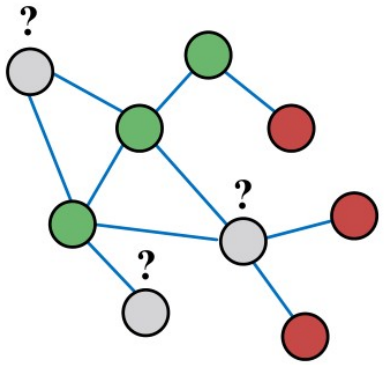
Similar nodes are typically close together or directly connected in the network:

- **Guilt-by-association**: If I am connected to a node with label X, then I am likely to have label X as well.
- **Example: Malicious/benign web page:** Malicious web pages link to one another to increase visibility, look credible, and rank higher in search engines

Classification label of a node v in network may depend on:

- Features of v
- Labels of the nodes in v's neighborhood
- Features of the nodes in v's neighborhood

Semi-supervised binary node classification
(частичное обучение для бинарной классификации год)



Given:
- Graph
- Few labeled nodes

Find: Class (red/green) of remaining nodes

Main assumption: There is homophily in the network

Example task:

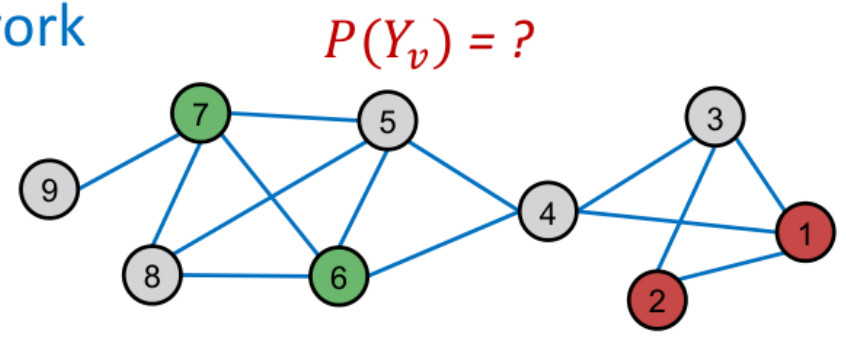
- Let $\boldsymbol{A}$ be a $n \times n$ adjacency matrix over n nodes
- Let $\mathrm{Y} = \{0, 1\}^n$ be a vector of **labels**:
  - $\mathrm{Y}_v = 1$ belongs to Class 1
  - $\mathrm{Y}_v = 0$ belongs to Class 0
  - There are unlabeled node needs to be classified
- **Goal:** Predict which **unlabeled** nodes are likely **Class 1**, and which are likely **Class 0**

Где применяется?

- Document classification
- Part of speech tagging
- Link prediction
- Optical character recognition
- Image/3D data segmentation
- Entity resolution in sensor networks
- Spam and fraud detection

Формулировка задачи

- **How to predict the labels Y_v for the unlabeled nodes v (in grey color)?**
- Each node v has a feature vector f_v
- Labels for some nodes are given (1 for green, 0 for red)
- **Task:** Find $P(Y_v)$ given all features and the network



Коллективная классификация

- **Intuition**: Simultaneous classification of interlinked nodes using correlations
- Probabilistic framework
- Markov Assumption: *the label Y_v of one node v depends on the labels of its neighbors N_v*

$$P(Y_v) = P(Y_v|N_v)$$

- Collective classification involves 3 steps:

Локальный классификатор – назначает метки нодам
Классификатор отношений – ищет корреляции между нодами
Коллективные вычисления – распространяет корреляции по графу

Локальный классификатор

Используется для начальной классификации нод.

- Предсказывает метку ноды основываясь на атрибутах/признаках ноды

- Не использует иноформацию о структуре графа

- Решается как стандартная задача классификации по признакам (решающее дерево, нейронная сеть и т.д.)

Связный классификатор (Relational classifier)

Используется для вычисления корреляций

- Предсказывает метку ноды основываясь на метках и атрибутах/признаках соседей этой ноды

- Использует иноформацию о структуре графа

Коллективные вычисления

Используется для иттертивного поиска устойчивого состояния результата (конверсии)

- Применяет иттеративно связный классификатор к каждой ноде

- Повторяет иттерации пока не достигнет состояния конферсии и прогноз не будет оптимален (минимизирует несоответствие между метками соседних мод)

- Информация о структуре графа влияет на результат

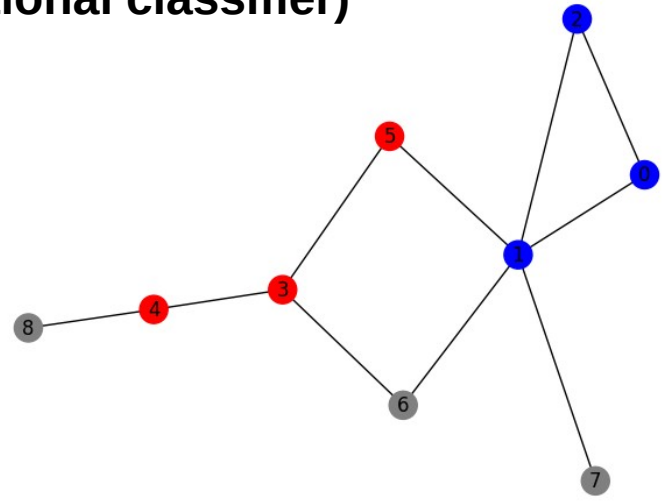
Связный классификатор
(Relational classifier)

Связный классификатор (Relational classifier)

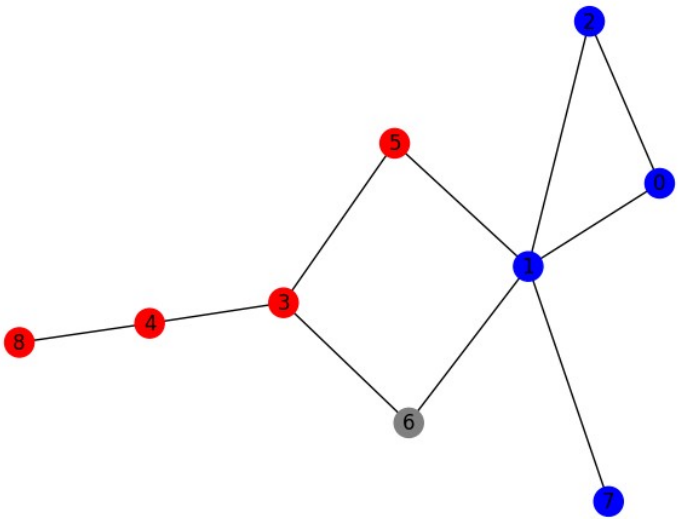
- **Idea: Propagate node labels across the network**
  - Class probability Y_v of node v is a weighted average of class probabilities of its neighbors.
- For **labeled nodes** v, initialize label Y_v with ground-truth label Y_v^*.
- For **unlabeled nodes**, initialize $Y_v = 0.5$.
- **Update** all nodes in a random order until convergence or until maximum number of iterations is reached.

Связный классификатор (Relational classifier)

```
G=nx.Graph()
G.add_nodes_from([(0,{'Y':1,'train':1}),(1,{'Y':1,'train':1}),(2,{'Y':1,'train':1}),
                  (3,{'Y':0,'train':1}),(4,{'Y':0,'train':1}),(5,{'Y':0,'train':1}),
                  (6,{'Y':-1,'train':0}),(7,{'Y':-1,'train':0}),(8,{'Y':-1,'train':0}),
                  ])
G.add_edges_from([(0,1),(0,2),(1,2),(3,4),(3,5),(3,6),(1,5),(1,6),(1,7),(4,8)])
cols={-1:'gray',0:'red',1:'blue'}
c = []
for node,attr in G.nodes.items():
    c.append(cols[attr['Y']])
pos=nx.kamada_kawai_layout(G)
nx.draw(G, pos,node_color=c, with_labels=True)
```




```
import random as rd
for node in G.nodes():
    if G.nodes[node]['train']==0: G.nodes[node]['Y']=0.5
for itt in range(1000):
 n=rd.choice(list(G.nodes()))
 w=[]
 for n_n in G.neighbors(n): w.append(G.nodes[n_n]['Y'])
 w=np.array(w)
 if(G.nodes[n]['train']==0): G.nodes[n]['Y']=w.mean()
c = []
for node,attr in G.nodes.items():
  if attr['Y']>0.5: c.append(cols[1])
  elif attr['Y']<0.5: c.append(cols[0])
  else: c.append('gray')
nx.draw(G, pos,node_color=c, with_labels=True)
```



Связный классификатор (Relational classifier)

Для взвешанного графа:

- **Update** for each node v and label c (e.g. 0 or 1)

$$P(Y_v = c) = \frac{1}{\sum_{(v,u)\in E} A_{v,u}} \sum_{(v,u)\in E} A_{v,u}\, P(Y_u = c)$$

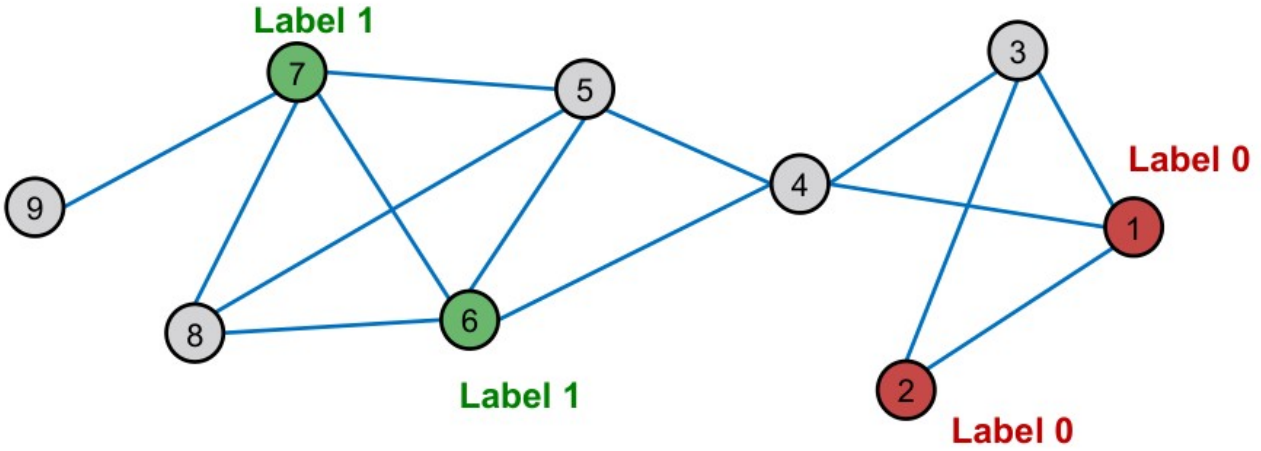
Суммирование по соседям

  - If edges have strength/weight information, $A_{v,u}$ can be the edge weight between v and u
  - $P(Y_v = c)$ is the probability of node v having label c

- Challenges:
  - Convergence is not guaranteed
  - Model cannot use node feature information

Связный классификатор (Relational classifier)

Задание: предсказать классы нод в графе:



Определить за сколько шагов алгоритм сходится

Иттеративный классификатор (Itterative classifier)

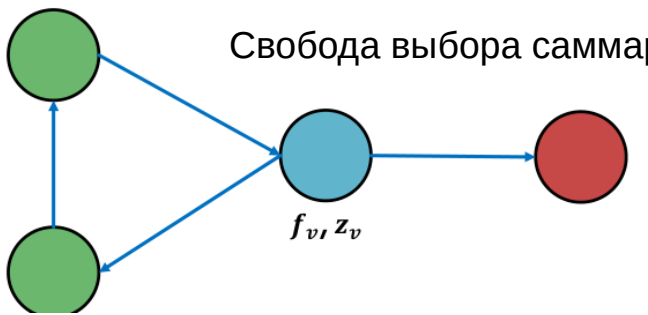
Иттеративный классификатор (Itterative classifier)

- **Relational classifier does not use node attributes.**
- **How can one leverage them?**
- **Main idea of iterative classification:** Classify node v based on its **attributes f_v** as well as **labels z_v** of neighbor set N_v.

- **Input: Graph**
  - f_v : feature vector for node v
  - Some nodes v are labeled with Y_v
- **Task:** Predict label of unlabeled nodes
- **Approach: Train two classifiers:**
  - $\phi_1(f_v)$ = Predict node label based on node feature vector f_v. This is called **base classifier.**
  - $\phi_2(f_v, z_v)$ = Predict label based on node feature vector f_v and summary z_v of labels of v's neighbors. This is called **relational classifier**.

How do we compute the summary z_v of labels of v's neighbors N_v?

- z_v = **vector that captures labels around node v**
  - Histogram of the number (or fraction) of each label in N_v
  - Most common label in N_v
  - Number of different labels in N_v



Иттеративный классификатор (Itterative classifier)

- **Phase 1: Classify based on node attributes alone**
  - On the labeled **training set**, train two classifiers:
    - **Base classifier:** $\phi_1(f_v)$ to predict Y_v based on f_v
    - **Relational classifier:** $\phi_2(f_v, z_v)$ to predict Y_v based on f_v and summary z_v of labels of v's neighbors
- **Phase 2: Iterate till convergence**
  - On **test set**, set labels Y_v based on the classifier ϕ_1, compute z_v and predict the labels with ϕ_2
  - **Repeat** for each node v:
    - Update z_v based on Y_u for all $u \in N_v$
    - Update Y_v based on the new z_v (ϕ_2)
  - Iterate until class labels stabilize or max number of iterations is reached
  - **Note:** Convergence is not guaranteed

Распространение доверия (belief propagation)

Распространение доверия (belief propagation)

- Belief Propagation is a dynamic programming approach to **answering probability queries in a graph** (e.g. probability of node v belonging to class 1)
- Iterative process in which neighbor nodes "talk" to each other, **passing messages**

"I (node v) believe you (node u) belong to class 1 with likelihood …"

- **When consensus is reached, calculate final belief**

Распространение доверия (belief propagation)

- **Task**: Count the number of nodes in a graph*
- **Condition:** Each node can only interact (pass message) with its neighbors

- **Example**: path graph

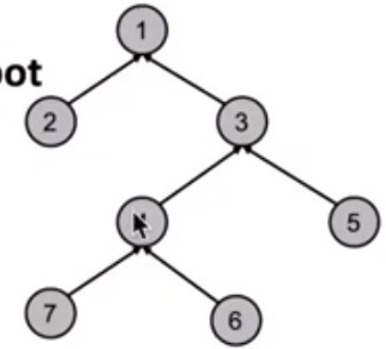
* Potential issues when the graph contains cycles.
We'll get back to it later!

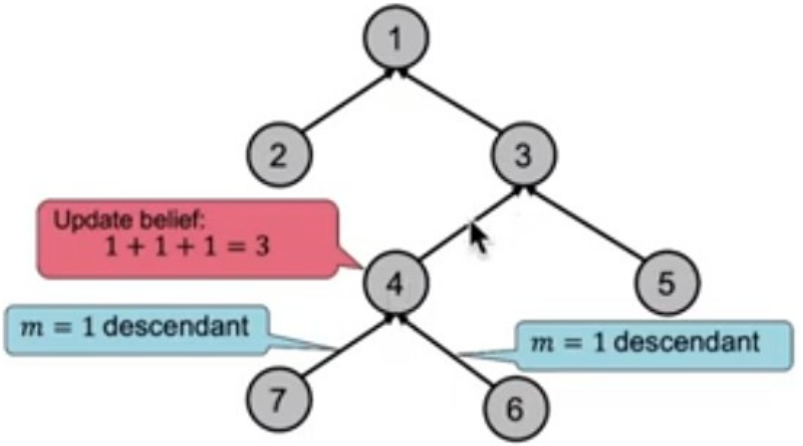
- **Task**: Count the number of nodes in a graph
- **Algorithm**:
  - Define an ordering of nodes (that results in a path)
  - Edge directions are according to order of nodes
    - Edge direction defines the order of message passing
  - For node i from 1 to 6
    - Compute the message from node i to $i+1$ (number of nodes counted so far)
    - Pass the message from node i to $i+1$

Task: Count the number of nodes in a graph
Condition: Each node can only interact (pass message) with its neighbors
Solution: Each node listens to the message from its neighbor, updates it, and passes it forward
m: the message

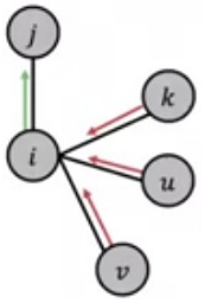
Распространение доверия (belief propagation)

- We can perform message passing not only on a path graph, but also on a tree-structured graph
- Define order of message passing from **leaves** to **root**





Распространение доверия (belief propagation)



What message will i send to j?

- It depends on what i hears from its neighbors
- Each neighbor passes a message to i its beliefs of the state of i

I (node i) believe that you (node j) belong to class Y_j with probability …

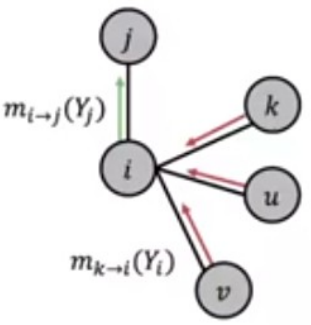
- ***Label-label potential matrix*** ψ : Dependency between a node and its neighbor. $\psi(Y_i, Y_j)$ is proportional to the probability of a node j being in class Y_j given that it has neighbor i in class Y_i.
- ***Prior belief*** ϕ : $\phi(Y_i)$ is proportional to the probability of node i being in class Y_i.
- $m_{i \to j}(Y_j)$ is i's message / estimate of j being in class Y_j.
- $\mathcal{L}$ is the set of all classes/labels

$$\psi(Y_i, Y_j) \propto P(state_j = Y_j | state_i = Y_i)$$

$$\phi(Y_i) \propto P(state_i = Y_i)$$

Распространение доверия (belief propagation)

- ***Label-label potential matrix*** ψ : Dependency between a node and its neighbor. $\psi(Y_i, Y_j)$ is proportional to the probability of a node j being in class Y_j given that it has neighbor i in class Y_i.
- ***Prior belief*** ϕ : $\phi(Y_i)$ is proportional to the probability of node i being in class Y_i.
- $m_{i \to j}(Y_j)$ is i's message / estimate of j being in class Y_j.
- $\mathcal{L}$ is the set of all classes/labels



1. Initialize all messages to 1
2. Repeat for each node:

$$m_{i \to j}(Y_j) = \sum_{Y_i \in \mathcal{L}} \psi(Y_i, Y_j)\, \phi_i(Y_i) \prod_{k \in N_i \setminus j} m_{k \to i}(Y_i) \quad \forall Y_j \in \mathcal{L}$$

Label-label potential

All messages sent by neighbors from previous round

Sum over all states

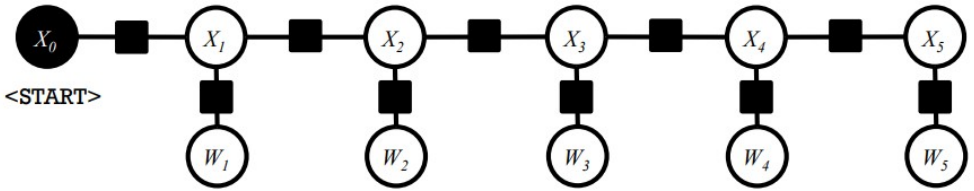
Prior

$$\psi(Y_i, Y_j) \propto P(state_j = Y_j | state_i = Y_i)$$

$$\phi(Y_i) \propto P(state_i = Y_i)$$

Распространение доверия (belief propagation)

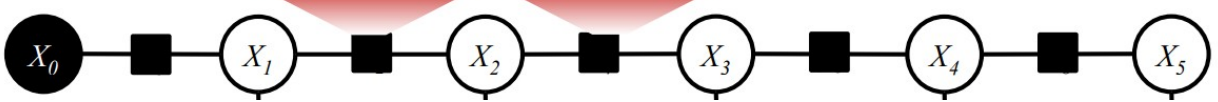
Sample 1:	n	v	p	d	n
	time	flies	like	an	arrow
Sample 2:	n	n	v	d	n
	time	flies	like	an	arrow
Sample 3:	n	v	p	n	n
	flies	fly	with	their	wings
Sample 4:	p	n	n	v	v
	with	time	you	will	see



$$\psi\left(state_j = Y_j \middle| state_i = Y_i\right)$$

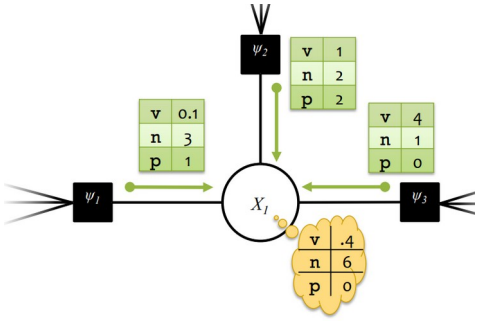
	v	n	p	d
v	1	6	3	4
n	8	4	2	0.1
p	1	3	1	3
d	0.1	8	0	0

	v	n	p	d
v	1	6	3	4
n	8	4	2	0.1
p	1	3	1	3
d	0.1	8	0	0



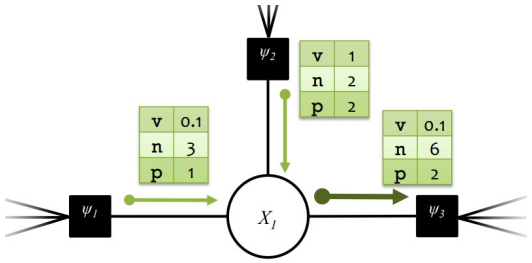
Note: We chose to reuse the same factors at different positions in the sentence.

Распространение доверия (belief propagation)



$$b_i(x_i) = \prod_{\alpha \in \mathcal{N}(i)} \mu_{\alpha \to i}(x_i)$$

1. Определение мнения о себе с точки зрения соседей

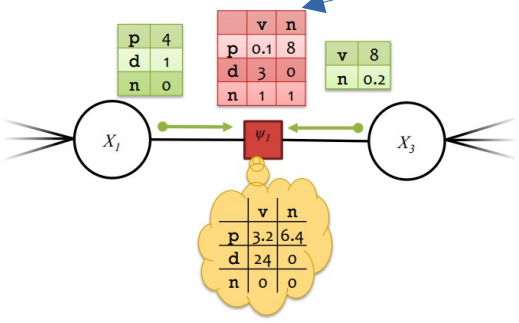


$$\mu_{i \to \alpha}(x_i) = \prod_{\alpha \in \mathcal{N}(i) \setminus \alpha} \mu_{\alpha \to i}(x_i)$$

2. Сообщение мнения других о себе соседям

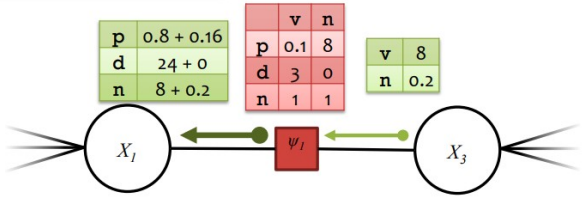
CRF

$$\psi(state_j = Y_j | state_i = Y_i)$$



$$b_\alpha(\boldsymbol{x_\alpha}) = \psi_\alpha(\boldsymbol{x_\alpha}) \prod_{i \in \mathcal{N}(\alpha)} \mu_{i \to \alpha}(\boldsymbol{x_\alpha}[i])$$

3. Своя оценка соседей по их сообщениям о себе



$$\mu_{\alpha \to i}(x_i) = \sum_{\boldsymbol{x_\alpha} : \boldsymbol{x_\alpha}[i] = x_i} \psi_\alpha(\boldsymbol{x_\alpha}) \prod_{j \in \mathcal{N}(\alpha) \setminus i} \mu_{j \to \alpha}(\boldsymbol{x_\alpha}[i])$$

4.Передача своего и чужого мнения о соседе

Распространение доверия (belief propagation)

Input: a factor graph with no cycles

Output: exact marginals for each variable and factor

Algorithm:

1. Initialize the messages to the uniform distribution.

$$\mu_{i\to\alpha}(x_i) = 1 \qquad \mu_{\alpha\to i}(x_i) = 1$$

2. Choose a root node.

3. Send messages from the **leaves** to the **root**.
   Send messages from the **root** to the **leaves**.

$$\mu_{i\to\alpha}(x_i) = \prod_{\alpha\in\mathcal{N}(i)\setminus\alpha} \mu_{\alpha\to i}(x_i) \qquad \mu_{\alpha\to i}(x_i) = \sum_{\boldsymbol{x}_\alpha:\boldsymbol{x}_\alpha[i]=x_i} \psi_\alpha(\boldsymbol{x}_\alpha) \prod_{j\in\mathcal{N}(\alpha)\setminus i} \mu_{j\to\alpha}(\boldsymbol{x}_\alpha[i])$$

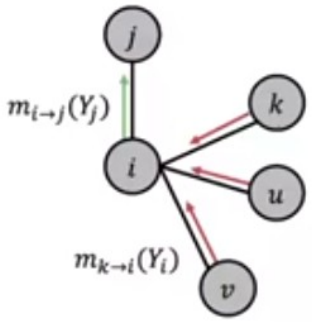
4. Compute the beliefs (unnormalized marginals).

$$b_i(x_i) = \prod_{\alpha\in\mathcal{N}(i)} \mu_{\alpha\to i}(x_i) \qquad b_\alpha(\boldsymbol{x}_\alpha) = \psi_\alpha(\boldsymbol{x}_\alpha) \prod_{i\in\mathcal{N}(\alpha)} \mu_{i\to\alpha}(\boldsymbol{x}_\alpha[i])$$

5. Normalize beliefs and return the **exact** marginals.

$$p_i(x_i) \propto b_i(x_i) \qquad p_\alpha(\boldsymbol{x}_\alpha) \propto b_\alpha(\boldsymbol{x}_\alpha)$$

Распространение доверия (belief propagation)



1. Initialize all messages to 1
2. Repeat for each node:

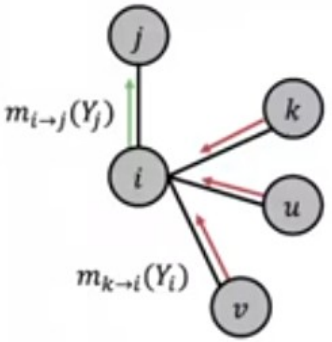
Label-label potential

All messages sent by neighbors from previous round

$$m_{i\to j}(Y_j) = \sum_{Y_i\in\mathcal{L}} \psi(Y_i, Y_j)\,\phi_i(Y_i) \prod_{k\in N_i\setminus j} m_{k\to i}(Y_i), \quad \forall Y_j \in \mathcal{L}$$

Sum over all states

Prior



After convergence:

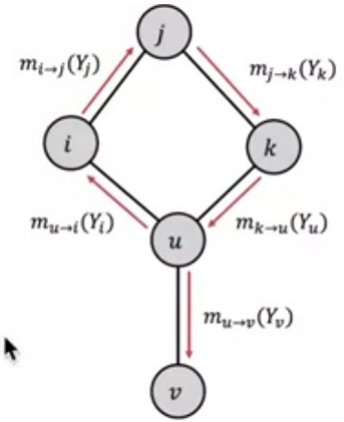
$b_i(Y_i)$ = node i's belief of being in class Y_i

Prior

All messages from neighbors

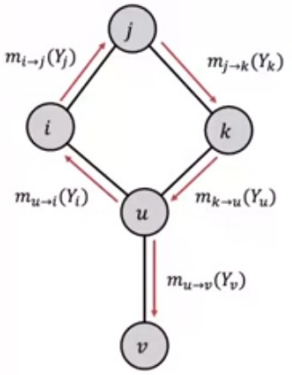
$$b_i(Y_i) = \phi_i(Y_i) \prod_{j\in N_i} m_{j\to i}(Y_j), \quad \forall\, Y_i \in \mathcal{L}$$

Распространение доверия (belief propagation)



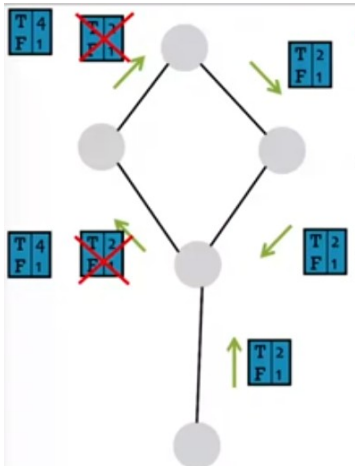
Messages from different subgraphs are **no longer independent**!

But we can still run BP, but it will pass messages in loops.



- **Beliefs may not converge**
  - Message $m_{u\to i}(Y_i)$ is based on initial belief of i, not a **separate evidence** for i
  - The initial belief of i (which could be incorrect) is reinforced by the cycle $i \to j \to k \to u \to i$
- However, in practice, Loopy BP is still a good heuristic for complex graphs which contain many branches.

- **Advantages**:
  - Easy to program & parallelize
  - General: can apply to any graph model with any form of potentials
    - Potential can be higher order: e.g. $\psi(Y_i, Y_j, Y_k, Y_v \ldots)$
- **Challenges**:
  - Convergence is not guaranteed (when to stop), especially if many closed loops
- **Potential functions** (parameters)
  - Require training to estimate



- Messages loop around and around: 2, 4, 8, 16, 32, ... More and more convinced that these variables are T!
- BP incorrectly treats this message as **separate evidence** that the variable is T (true).
- Multiplies these two messages as if they were independent.
  - But they don't actually come from *independent* parts of the graph.
  - One influenced the other (via a cycle).

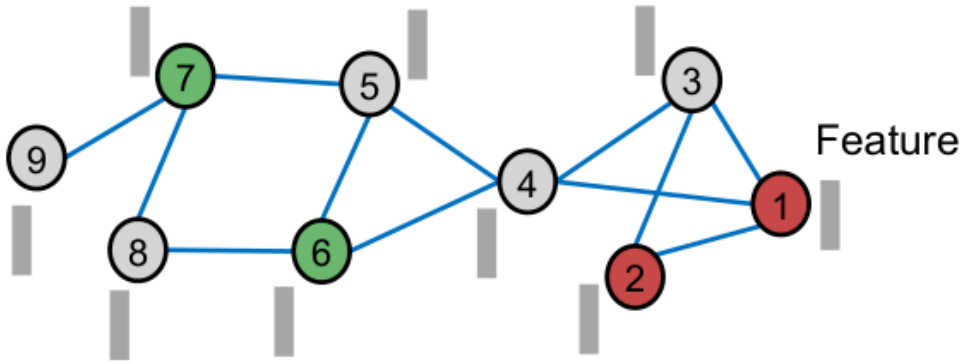
$$m^k_{t\to s}(x_s) = \lambda m_{t\to s}(x_s) + (1-\lambda) m^{k-1}_{t\to s}(x_s)$$

Ослабление сообщения

Correct & Smooth

Correct & Smooth

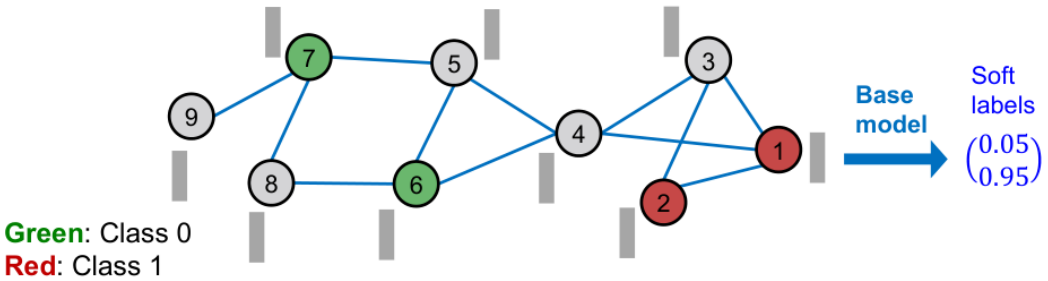
- **Setting:** A partially labeled graph and features over nodes.



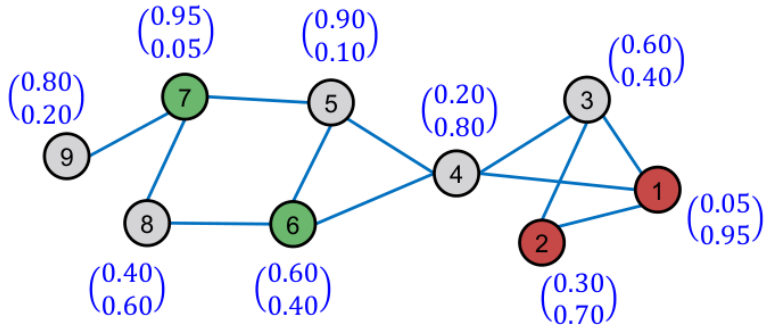
- **C&S follows the three-step procedure:**
  1. Train base predictor
  2. Use the base predictor to predict soft labels of all nodes.
  3. **Post-process the predictions using graph structure** to obtain the final predictions of all nodes.

Correct & Smooth

- **(1) Train a base predictor that predict soft labels (class probabilities) over all nodes.**
  - Labeled nodes are used for train/validation data.
  - Base predictor can be simple:
    - Linear model/Multi-Layer-Perceptron(MLP) over node features

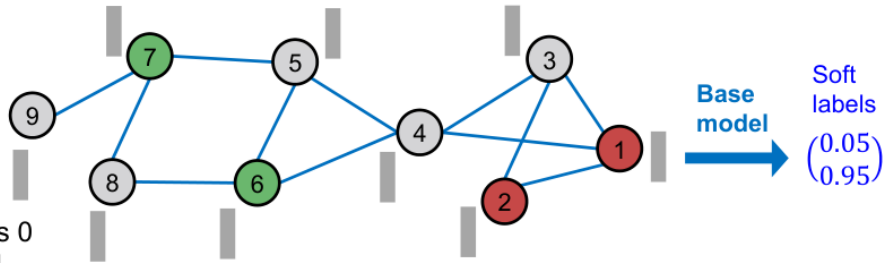


- **(2) Given a trained base predictor, we apply it to obtain soft labels for all the nodes.**
  - We expect these soft labels to be decently accurate.
  - **Can we use graph structure to post-process the predictions to make them more accurate?**



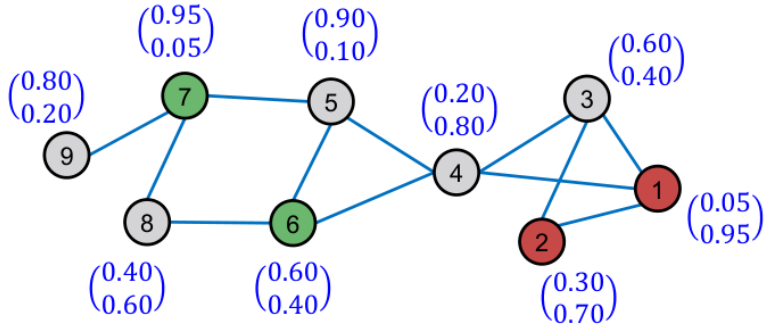
Correct & Smooth

- **(1) Train a base predictor that predict soft labels (class probabilities) over all nodes.**
  - Labeled nodes are used for train/validation data.
  - Base predictor can be simple:
    - Linear model/Multi-Layer-Perceptron(MLP) over node features



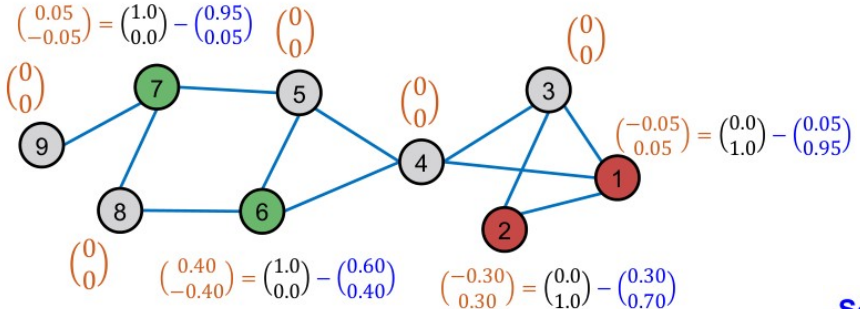
Green: Class 0
Red: Class 1

- **(2) Given a trained base predictor, we apply it to obtain soft labels for all the nodes.**
  - We expect these soft labels to be decently accurate.
  - **Can we use graph structure to post-process the predictions to make them more accurate?**



Correct & Smooth

- **Correct step:** ***Диффузия ошибки***
  - Compute training errors of nodes.
    - **Training error:** Ground-truth label minus soft label. Defined as 0 for unlabeled nodes.



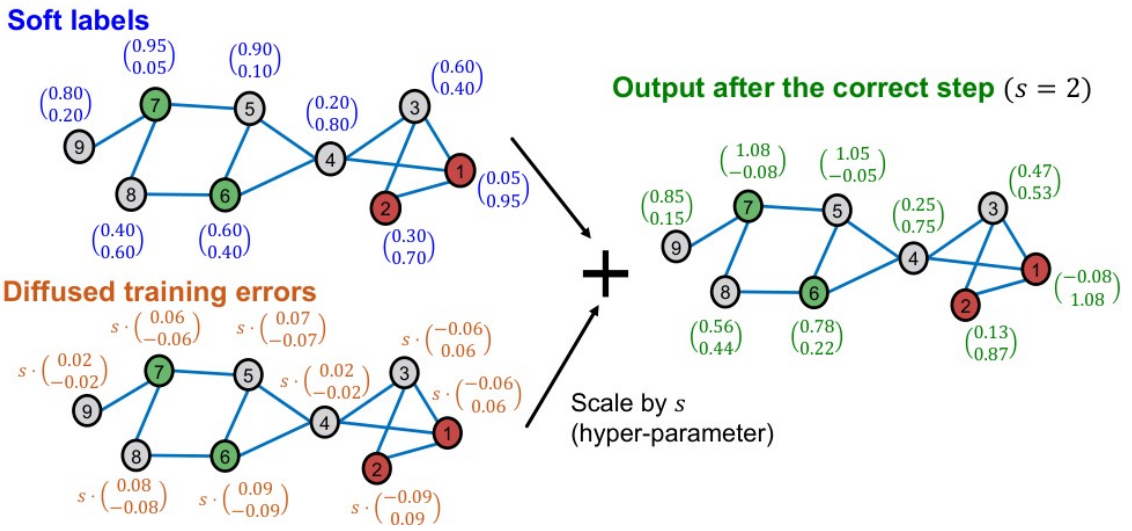
$$\boldsymbol{E}^{(0)} = \begin{pmatrix} -0.05 & 0.05 \\ -0.30 & 0.30 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0.40 & -0.40 \\ 0.05 & -0.05 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

Hyper-parameter

$$\boldsymbol{E}^{(t+1)} \leftarrow (1-\alpha)\cdot\boldsymbol{E}^{(t)} + \alpha\cdot\widetilde{\boldsymbol{A}}\boldsymbol{E}^{(t)}.$$

$$\widetilde{\boldsymbol{A}} \equiv \boldsymbol{D}^{-1/2}\boldsymbol{A}\boldsymbol{D}^{-1/2}$$

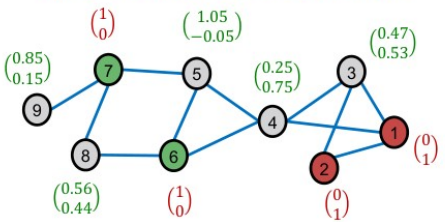
$\boldsymbol{D} \equiv \mathrm{Diag}(d_1, \ldots, d_N)$ be the degree matrix.



Correct & Smooth

- **Smoothen the corrected soft labels along the edges.**
  - **Assumption**: Neighboring nodes tend to share the same labels.
  - **Note:** For training nodes, we use the **ground-truth hard labels** instead of the soft labels.

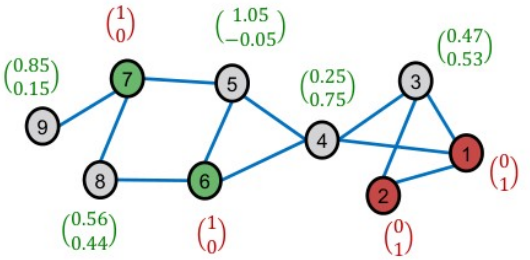
Input to the smooth step:



- **Smooth step:** ***Диффузия меток***
  - Diffuse label $Z^{(0)}$ along the graph structure.
  - $Z^{(t+1)} \leftarrow (1-\alpha) \cdot Z^{(t)} + \alpha \cdot \tilde{A}Z^{(t)}$.

Hyper-parameter

Diffuse labels along the edges

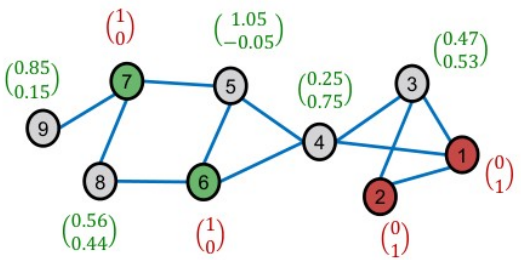


Corrected label matrix

$$Z^{(0)} = \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 0.47 & 0.53 \\ 0.25 & 0.75 \\ 1.05 & -0.05 \\ 1 & 0 \\ 1 & 0 \\ 0.56 & 0.44 \\ 0.85 & 0.15 \end{pmatrix}$$

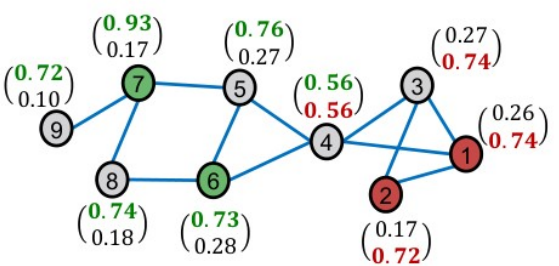
Before smoothing

$Z^{(0)}$

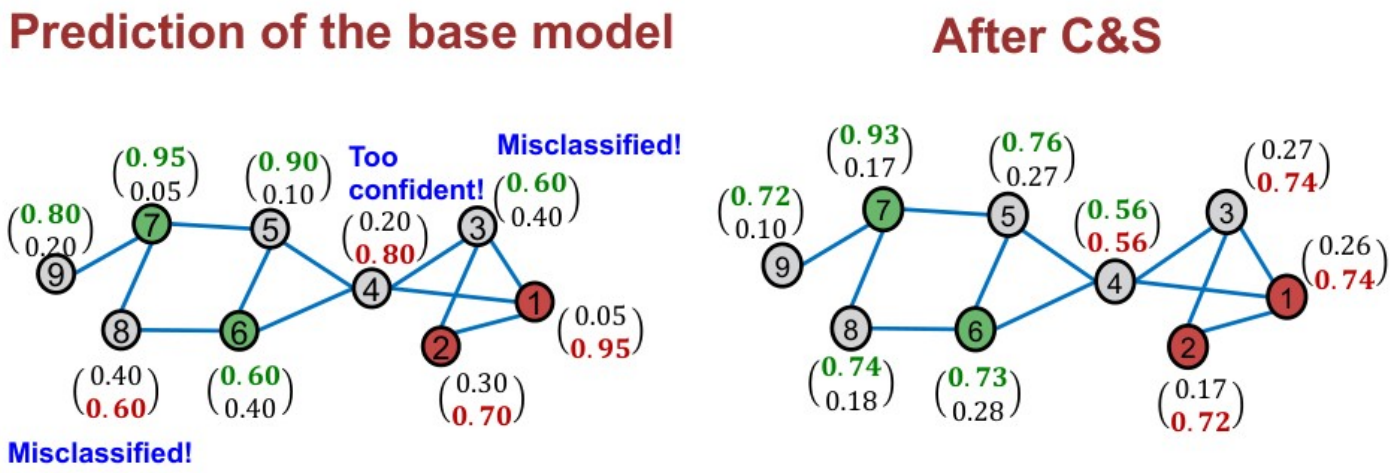


After smoothing

$Z^{(3)}, \alpha = 0.8$



Correct & Smooth



https://ogb.stanford.edu/docs/nodeprop/#ogbn-arxiv

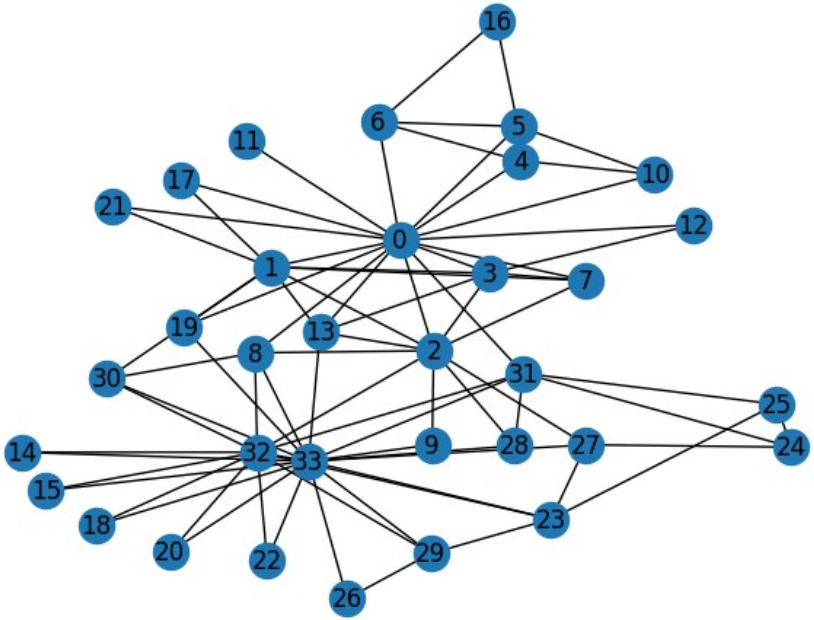
Loading datasets using torch_geometric

Loading datasets using torch_geometric

```
from torch_geometric.datasets import KarateClub
import torch_geometric
import networkx as nx

dataset = KarateClub()
data = dataset[0]

g = torch_geometric.utils.to_networkx(data, to_undirected=True)
pos=nx.kamada_kawai_layout(g)
nx.draw(g,pos,with_labels=True)
```



Complex datasets

Cora is the most popular dataset for node classification in the scientific literature. It represents a network of 2,708 publications, where each connection is a reference. Each publication is described as a binary vector of 1,433 unique words, where 0 and 1 indicate the absence or presence of the corresponding word, respectively.

The goal is to classify each node into one of seven categories.

```
import torch_geometric
import networkx as nx
from torch_geometric.datasets import Planetoid
dataset = Planetoid(root=".", name="Cora")
data = dataset[0]
print(f'Dataset: {dataset}')
print('---------------')
print(f'Number of graphs: {len(dataset)}')
print(f'Number of nodes: {data.x.shape[0]}')
print(f'Number of features: {dataset.num_features}')
print(f'Number of edges: {dataset.edge_index.shape[1]}')
print(f'Number of classes: {dataset.num_classes}')
print(f'Edges are directed: {data.is_directed()}')
print(f'Graph has isolated nodes: {data.has_isolated_nodes()}')
print(f'Graph has loops: {data.has_self_loops()}')
```

```
g = torch_geometric.utils.to_networkx(data, to_undirected=True)
pos=nx.spring_layout(g)
nx.draw(g,node_size=1)
```

```
Dataset: Cora()
---------------
Number of graphs: 1
Number of nodes: 2708
Number of features: 1433
Number of edges: 10556
Number of classes: 7
Edges are directed: False
Graph has isolated nodes: False
Graph has loops: False
```